

# DataDignity

## FINAL PROJECT STRUCTURE & PHASE-BY-PHASE EXECUTION PLAN

**Project:** Zero-Knowledge Digital Estate Plan & Post-Mortem Privacy Infrastructure

**Core principle:** Build the system security-first, with key recovery, beneficiary authenticity, revocation, liveness verification, asset classification, and legal limitations designed before irreversible automation.

### Phase 0 — Project Foundation, Threat Model & Architecture

- Finalize functional requirements and system boundaries.
- Define the zero-knowledge trust model and threat model.
- Define the complete vault/key lifecycle and beneficiary recovery architecture.
- Define account lifecycle states, warning stages, Legacy/Scrub behavior, and default handling for unclassified assets.
- Establish repository structure, development conventions, module boundaries, and interfaces.

**Deliverable:** Complete architecture, threat model, key lifecycle, lifecycle/state model, and technical decision record.

### Phase 1 — Application Shell & UI Foundation

- Build application shell, routing, authentication UI, and protected workspace.
- Create dashboard, vault, beneficiary, check-in, warning, trusted-contact, and protocol-status interfaces.
- Implement the DataDignity visual language: slate structure, white cards, muted emerald, cool blue, and soft lavender accents.
- Make consent, warnings, and irreversible actions visually explicit.

**Deliverable:** Complete navigable frontend with all major user workflows represented.

### Phase 2 — Client-Side Cryptography & Key Management

- Implement Web Crypto API and AES-GCM-256 encryption/decryption.
- Implement secure IVs, salts, key derivation, and a separate Data Encryption Key (DEK).
- Define DEK wrapping/unwrapping and browser-side key handling.
- Require a beneficiary recovery design that does not depend on the unavailable user's passphrase.
- Preferred approach: beneficiary public keys are registered while the user is active and the DEK is wrapped separately for each authorized beneficiary.
- Provide beneficiary key-health reminders plus a backup/recovery path for lost beneficiary keys.
- Test encryption, tampering, invalid keys, plaintext leakage, recovery, and lost-key scenarios.

**Deliverable:** Production-grade client cryptography and a recoverable beneficiary key architecture.

### Phase 3 — Vault, Database & Identity Unlinkability

- Implement PostgreSQL and Prisma models.
- Separate identity information, identity-to-vault mapping, and encrypted payload storage.
- Use pseudonymous vault IDs if unlinkability is a project goal.
- Protect the mapping layer with separate access controls.
- Store ciphertext and wrapped key material rather than plaintext.
- Implement authorization, ownership checks, and tenant isolation.
- Test whether database access allows trivial identity-to-vault correlation.

**Deliverable:** Ciphertext-only vault infrastructure with controlled identity correlation.

### Phase 4 — Vault Management & Beneficiary Onboarding

- Create vault entry creation, editing, deletion, categorization, and beneficiary assignment.
- Require beneficiaries to onboard before Legacy-eligible assets become active.
- Register beneficiary public keys and generate cryptographic fingerprints.
- Require an out-of-band fingerprint verification step before trusting a beneficiary key.
- Support adding, removing, replacing, and re-verifying beneficiaries.
- When beneficiary membership changes, rotate the DEK where required and re-wrap it for remaining beneficiaries.
- Test fake keys, MITM registration scenarios, revocation, rotation, and cross-user isolation.

**Deliverable: Every Legacy-eligible asset has a verified and recoverable beneficiary path.**

## Phase 5 — Liveness, Check-In, OAuth & Trusted Contact

- Make a dedicated user-controlled check-in the primary liveness signal.
- Use OAuth activity as secondary corroboration, not as proof of death/inactivity.
- Use minimum OAuth scopes and handle token expiry, revocation, provider outages, and duplicate signals.
- Add a designated trusted contact who is notified during the warning window.
- Use human escalation as an additional safeguard without treating the trusted contact as automatic legal executor or proof of death.

**Deliverable: Multi-signal liveness framework with human escalation and provider-failure handling.**

## Phase 6 — Warning System, State Machine & Cron Engine

- Implement staged inactivity states rather than immediate Legacy/Scrub triggering.
- Example default: Day 60 warning, Day 75 warning, Day 85 final warning, Day 90+ trigger eligibility.
- Send progressively urgent notifications.
- Provide explicit cancellation/reactivation.
- Prefer active user response as the strongest cancellation signal.
- Implement scheduled jobs, lifecycle transitions, locking, idempotency, retries, recovery, and duplicate-trigger prevention.
- Require a final eligibility/safety gate before irreversible actions.

**Deliverable: Reliable staged inactivity state machine that reduces false-trigger risk.**

## Phase 7 — Asset Classification & Policy Engine

- Define explicit asset states: LEGACY, SCRUB, and UNCLASSIFIED.
- LEGACY assets become eligible for beneficiary recovery.
- SCRUB assets become eligible for deletion.
- UNCLASSIFIED assets receive no irreversible automatic action.
- An asset must never execute both Legacy and Scrub.
- Conflicting metadata enters SAFE HOLD and requires explicit resolution.
- User configuration takes precedence over inferred/default behavior.

**Deliverable: Deterministic asset-policy engine with safe defaults and conflict handling.**

## Phase 8 — Legacy Protocol

- Verify the asset is classified as LEGACY.
- Verify trigger eligibility and beneficiary authorization.
- Verify beneficiary key status.
- Deliver ciphertext plus the appropriate beneficiary-wrapped DEK.
- Allow the beneficiary to unwrap/decrypt locally.

- Never decrypt the asset on the server.
- Handle key loss, revoked beneficiaries, invalid keys, failed delivery, duplicate delivery, and corrupted ciphertext.

**Deliverable: End-to-end Legacy workflow in which a beneficiary can recover authorized data without the original user's passphrase.**

## Phase 9 — Scrub Protocol

- Verify the asset is classified as SCRUB and all final safety conditions are satisfied.
- Request deletion through supported third-party APIs where authorization is valid.
- Explicitly handle expired/revoked OAuth tokens, provider outages, insufficient permissions, and unsupported APIs.
- Distinguish DELETION\_REQUESTED, DELETION\_CONFIRMED, DELETION\_FAILED, and DELETION\_UNAVAILABLE.
- Never claim third-party deletion succeeded without confirmation.
- Purge DataDignity records according to the defined retention/deletion policy.
- Make deletion jobs idempotent, auditable, and protected against accidental triggering.

**Deliverable: Controlled Scrub workflow with explicit provider limitations and truthful deletion states.**

## Phase 10 — Incremental Security & Reliability Testing

- After Phase 2: test cryptography, DEK recovery, plaintext exposure, and beneficiary-key attacks.
- After Phase 3: test identity/vault correlation, authorization, database compromise, and tenant isolation.
- After Phase 4: test fake beneficiary keys, MITM registration, fingerprint verification, revocation, and DEK rotation.
- After Phase 5: test OAuth manipulation, token expiry, provider outages, heartbeat spoofing, and trusted-contact escalation.
- After Phase 6: test false triggers, missed notifications, race conditions, duplicate jobs, cancellation, and state transitions.
- Run final tests for XSS, CSRF, injection, replay, authentication, session security, API abuse, dependency vulnerabilities, and secret/log leakage.

**Deliverable: Continuous security evidence throughout development plus a final threat-model review.**

## Phase 11 — Integration, Deployment & Production Readiness

- Integrate authentication, vault, encryption, beneficiary setup, key verification, check-in, heartbeat, warnings, trusted contact, policy engine, Legacy, and Scrub.
- Run complete end-to-end tests using synthetic data.
- Configure production secrets, PostgreSQL, background jobs, monitoring, alerts, rate limits, backups, recovery, rollback, and incident response.
- Keep irreversible actions disabled until all required security gates pass.

**Deliverable: Integrated production-ready DataDignity system.**

## Phase 12 — Final Demo, Documentation & Evaluation

- Document architecture, data flow, trust boundaries, and database structure.
- Document client-side encryption, DEK lifecycle, beneficiary wrapping, recovery, fingerprint verification, and revocation/rotation.
- Document check-in, OAuth corroboration, warning stages, trusted-contact escalation, and asset classification.
- Document Legacy, Scrub, failure states, and known limitations.
- Demonstrate normal Legacy recovery, false-trigger cancellation, beneficiary key recovery, OAuth failure, and Scrub provider failure using synthetic data.
- Include the legal limitation: automated inactivity detection is not legal proof of death and does not establish executor authority, probate authority, or legal entitlement to access or destroy assets; DataDignity is not a substitute for applicable estate-planning or probate processes.
- Finalize README, technical documentation, test results, and presentation.

**Deliverable: Complete project package suitable for evaluation, demonstration, and future development.**

## Final Subagent Structure

| Subagent                     | Responsibility                                                                   | Main Phases |
|------------------------------|----------------------------------------------------------------------------------|-------------|
| A — Architect / Threat Model | Architecture, trust model, threat model, contracts, lifecycle and security gates | 0, 10       |
| B — Frontend                 | Dashboard, vault, beneficiary, warnings, check-in and consent UX                 | 1, 4, 5, 6  |
| C — Crypto / Key Management  | Web Crypto, DEK, key wrapping, recovery, fingerprinting and rotation             | 2, 4, 8     |
| D — Backend / Database       | API, PostgreSQL, Prisma, authorization and unlinkability                         | 3, 4        |
| E — Liveness / OAuth         | Check-ins, OAuth corroboration, provider handling and trusted contact            | 5           |
| F — Automation               | State machine, warnings, cron, retries and trigger gates                         | 6           |
| G — Policy Engine            | LEGACY/SCRUB/UNCLASSIFIED rules and conflict resolution                          | 7           |
| H — Legacy Protocol          | Beneficiary delivery and local recovery                                          | 8           |
| I — Scrub Protocol           | External deletion, provider limitations and DataDignity purge                    | 9           |
| J — Security / QA            | Incremental threat testing, integration testing and final security review        | 2–12        |
| K — DevOps                   | Deployment, monitoring, secrets, backups and recovery                            | 11          |

## Critical Design Decisions to Lock Early

1. Beneficiary recovery: pre-register beneficiary public keys and wrap the DEK to them while the user is active; use threshold recovery only if the project explicitly requires it.
2. Beneficiary authenticity: verify public-key fingerprints through an out-of-band channel before trusting the key.
3. Beneficiary key loss: provide key-health reminders and a defined backup/recovery or key-replacement path.
4. Revocation: rotate the DEK and re-wrap it for remaining beneficiaries when true revocation is required.
5. Liveness: dedicated check-ins are primary; OAuth activity is secondary corroboration and provider failure must not imply death.
6. Human escalation: trusted contact participates during the warning window but is not automatically legal executor or proof of death.
7. Asset policy: LEGACY and SCRUB are mutually exclusive; UNCLASSIFIED assets receive no irreversible automatic action; conflicts enter SAFE HOLD.
8. Unlinkability: if required, use pseudonymous vault IDs and a separately protected identity-to-vault mapping rather than a directly joinable user\_id alone.
9. Scrub reality: distinguish requested, confirmed, failed, and unavailable third-party deletion.
10. Legal boundary: automated inactivity detection does not establish death, executor authority, probate authority, or legal entitlement.

## Final Execution Order

**Architecture → UI Foundation → Crypto + DEK + Beneficiary Recovery → Vault/Database + Unlinkability → Beneficiary Onboarding + Key Verification → Check-In + OAuth + Trusted Contact → Warning/State Machine + Cron → Asset Policy Engine → Legacy → Scrub → Continuous + Final Security Testing → Integration/Deployment → Demo/Documentation.**

## Milestone Gates

- Gate 1: Architecture, threat model, lifecycle, asset policy, and key-recovery model approved.
- Gate 2: Client encryption, DEK handling, beneficiary recovery, and key-health design proven.
- Gate 3: Ciphertext-only backend and identity/vault separation verified.
- Gate 4: Beneficiary onboarding, fingerprint verification, revocation, and DEK rotation verified.
- Gate 5: Check-in, OAuth corroboration, and trusted-contact escalation verified.
- Gate 6: Warning, cancellation, and inactivity state machine verified.

- Gate 7: Legacy workflow verified with synthetic data.
- Gate 8: Scrub workflow verified with explicit provider failure states.
- Gate 9: End-to-end security and reliability testing passed.
- Gate 10: Deployment, monitoring, documentation, legal limitations, and final demo complete.